



Introduction.

outlines

- Background and definitions of security
- Security Architecture
- Security Functional Requirements
- Fundamental Security Design Principles
- Standards in Security Techniques and Applications

Introduction

The art of war teaches us to rely not on the likelihood of the enemy's not coming, but on our own readiness to receive him; not on the chance of his not attacking, but rather on the fact that we have made our position unassailable.

—***The Art of War*, Sun Tzu**

Problem Definition – Why Security?

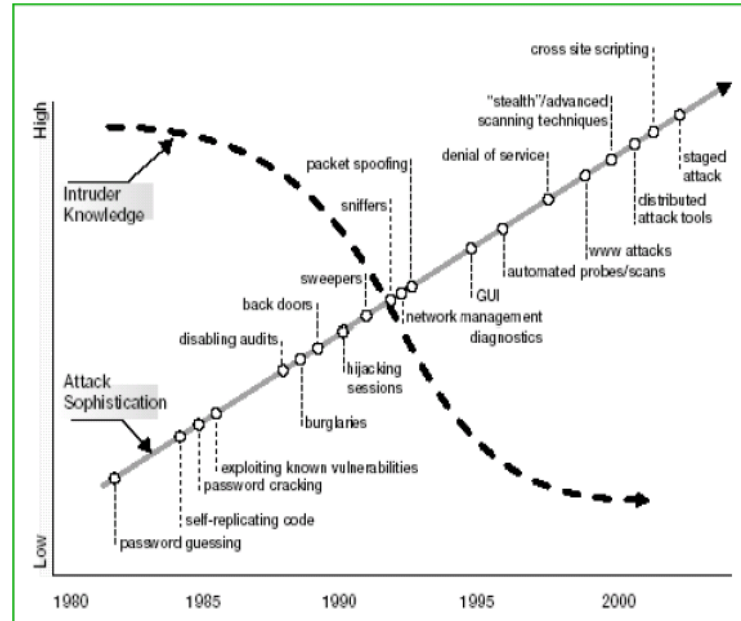
Evolution of technology focused
on ease of use

Moving the ball towards ease of
use means moving away from
security and functionality

Increased network environment
and network based applications



Decreasing scale level needed for exploits



Background

- Information Security requirements have changed in recent times
- traditionally provided by physical and administrative mechanisms
- computer use requires automated tools to protect files and other stored information
- use of networks and communications links requires measures to protect data during transmission

- **Computer Security:** Measures and controls that ensure confidentiality, integrity, and availability of information system assets including hardware, software, firmware, and information being processed, stored, and communicated. i.e. **Computer Security:** generic name for the collection of tools designed to protect data and to thwart hackers.
- **Network Security:** measures to protect data during their transmission over a network.
- **Internet Security:** measures to protect data during their transmission over a collection of interconnected networks
- Internet security consists of measures to deter, prevent, detect, and correct security violations that involve the transmission & storage of information.

Security Attacks

Threat

- An action or event that might compromise security.
- A threat is a potential violation of security.

Vulnerability

- Existence of weakness, Design or implementation error that may lead to undesired event compromising the security of the system.

Attack

- An attack is any action that violates vulnerability.
- **Security Attack:** Any action that compromises the security of information.

Security Attacks

There are two
types of Attack

- A. *Passive Attacks*
- B. *Active Attacks*

Passive attack

- ☐ Monitoring of transmissions, attempt to know *who communicate with whom* and *message contents*. doesn't affects the system resources.
- ☐ Passive attacks are breaches of secrecy (confidentiality).

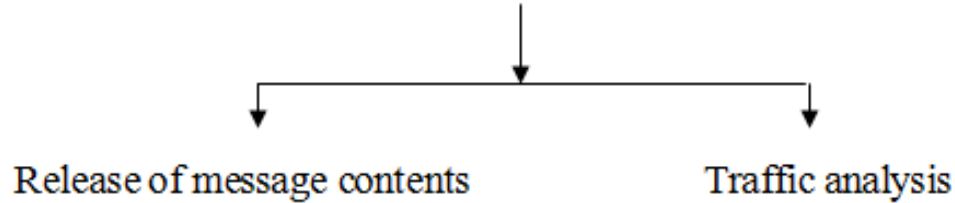
Active attack

- ☐ Attempt to alter system resources or affect their operation.
- ☐ Active attacks affect the availability, integrity, and authentication of data.

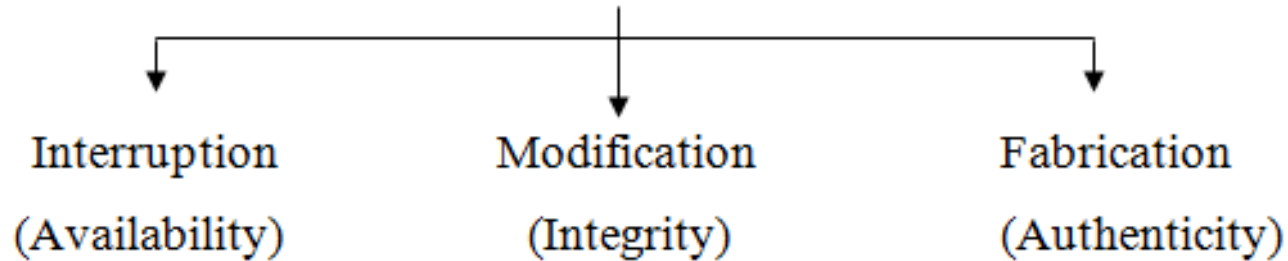
Security Attacks

Passive Attacks

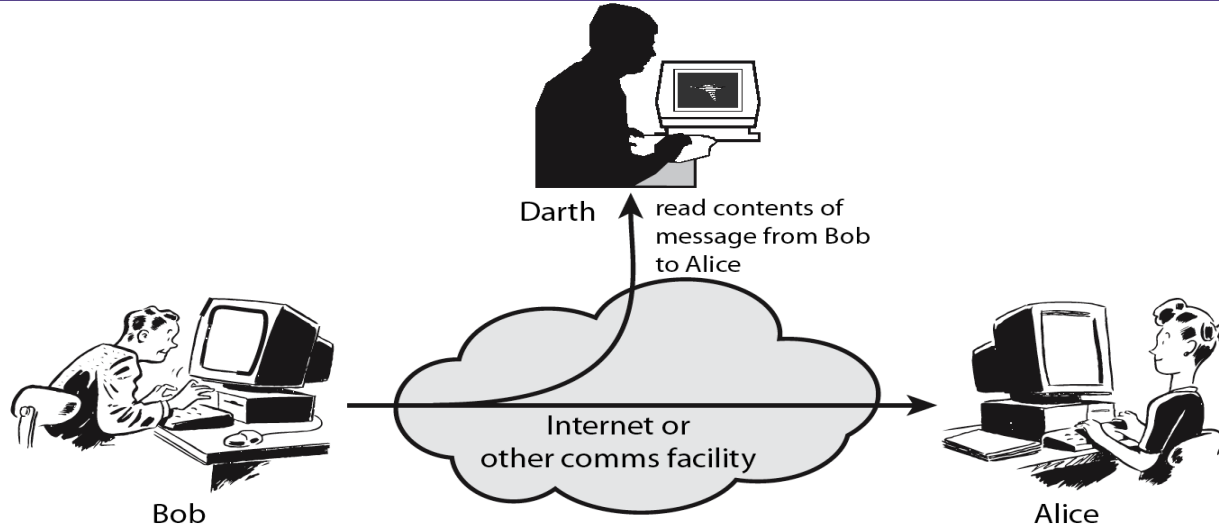
Interception (confidentiality)



Active Attacks



Security Attacks



- ❑ An unauthorized party gains access to an asset.
 - Release of message contents.
 - Traffic analysis.
- ❑ Passive attacks are breaches of confidentiality (secrecy).

Active attacks

1. Interruption/denial of service

- ☐ An asset of the system is destroyed or becomes unavailable.

Ex: destruction of a piece of hardware, such as a hard disk, or cutting of a communication line, destroy the bridge "wireless".

- ☐ This is an attack on availability.

2. Modification /replay/Masquerade

- ☐ An unauthorized party not only gains access to some portion of transmitted messages but also alters or reorders those messages to produce an unauthorized effect.

Ex: changing values in a data file, altering a program so that it performs differently.

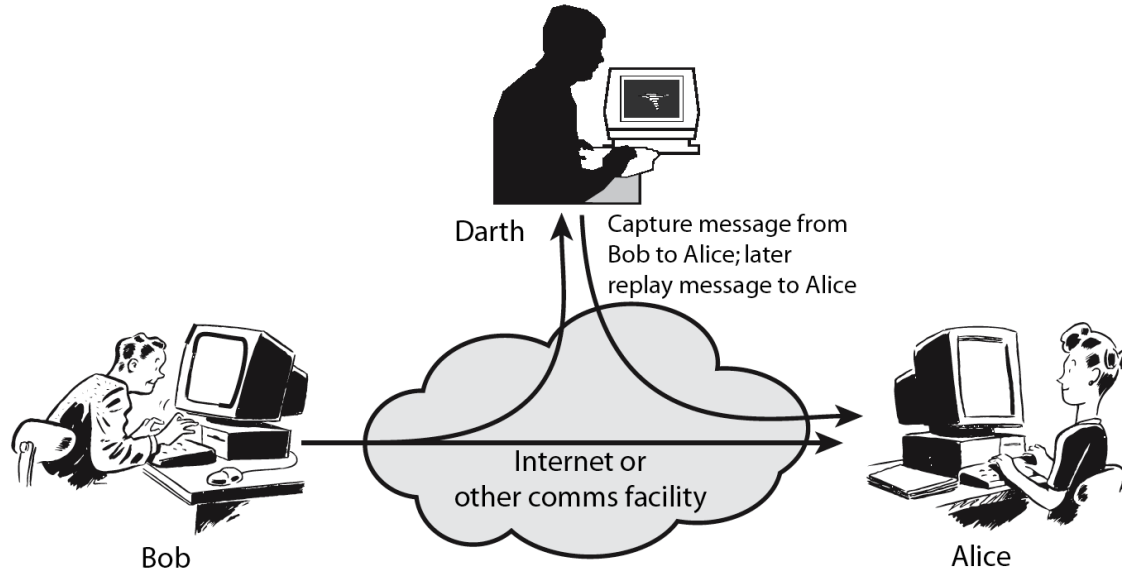
- ☐ This is an attack on integrity.

3. Fabrication/Masquerade

- ☐ An unauthorized party inserts false object into the system.

Ex: include the insertion of spurious messages in a network or the addition of records to a file.

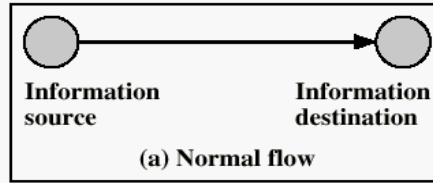
- ☐ This is an attack on authenticity.



- ❑ Active attacks affect the availability, integrity, and authentication of data.

Security Attacks

Interruption: This is an attack on availability



Interception: This is an attack on confidentiality

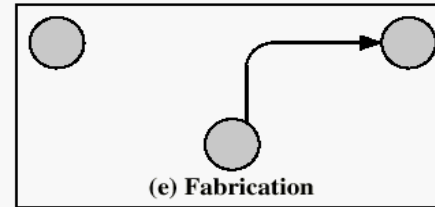
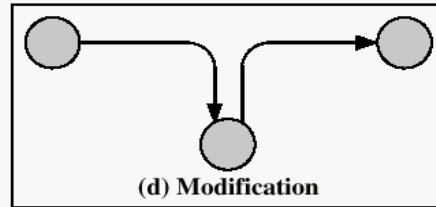
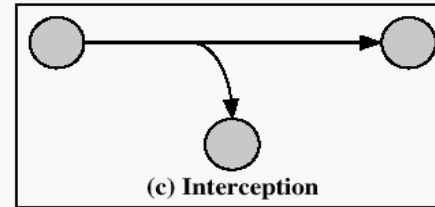
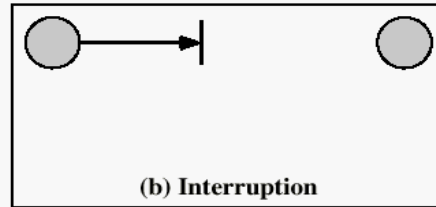


Figure 1.1 Security Threats

Modification: This is an attack on integrity

Fabrication: This is an attack on authenticity

Standards in Security Techniques and Applications:

- Importance of Standards:
 - Standards ensure consistency, reliability, and interoperability in security practices and technologies.
 - They are developed and promoted by various organizations to address global security challenges.
- Key Organizations Developing Security Standards:
 - National Institute of Standards and Technology (NIST):
 - A U.S. federal agency focused on measurement science, standards, and technology.
 - NIST's Federal Information Processing Standards (FIPS) and Special Publications (SP) have a global impact.
 - ✓ Example: NIST SP 800-53 provides guidelines for securing federal information systems.
- Internet Society (ISOC):
 - ✓ A professional membership society with global membership.
 - ✓ Oversees organizations like the Internet Engineering Task Force (IETF) and Internet Architecture Board (IAB), which develop Internet standards.
- Standards are published as Requests for Comments (RFCs).
 - Example: RFC 8446 defines the TLS 1.3 protocol for secure communication.
 - ITU-T (International Telecommunication Union - Telecommunication Standardization Sector):

Standards in Security Techniques and Applications:

- A United Nations agency coordinating global telecom networks and services.
- Produces standards known as Recommendations.
 - Example: ITU-T X.509 standard defines public key infrastructure (PKI) for digital certificates.
- International Organization for Standardization (ISO):
 - A worldwide federation of national standards bodies from over 140 countries.
 - Develops International Standards to facilitate global trade and cooperation.
- A list of ISO and NIST documents referenced in the book is included at the end.
- Examples of Standards in Action:
 - NIST: Used by U.S. government agencies and private sector organizations to implement cybersecurity frameworks.
 - ISOC/IETF: Internet standards like HTTP, HTTPS, and DNS are widely adopted globally.
 - ITU-T: Standards like X.509 are foundational for secure communication and digital identity.
 - ISO: ISO/IEC 27001 is a globally recognized standard for information security management.
 - Example: ISO/IEC 27001 specifies requirements for information security management systems (ISMS).

- ITU-T X.800 “Security Architecture for OSI”
- defines a systematic way of defining and providing security requirements for us it provides a useful, if abstract, overview of concepts we will study.



Security services

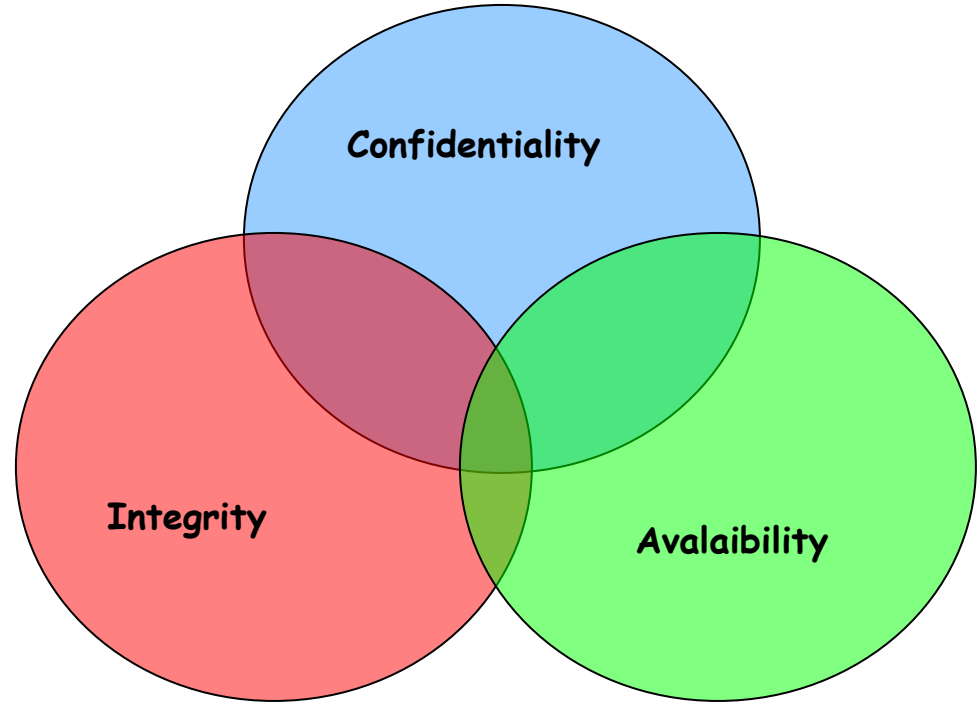
- **Security Service:** A service that enhances the security of data processing systems and information transfers. A security service makes use of one or more security mechanisms.
- X.800 defines the security service:
 - “a service provided by a protocol layer of communicating open systems, which ensures adequate security of the systems or of data transfers”
- RFC 2828 defines the security service:
 - “a processing or communication service provided by a system to give a specific kind of protection to system resources”
- **Security Service** enhances the security of data processing systems and information transfers of an organization. It is intended to counter security attacks. It is using one or more security mechanisms.
- It often replicates functions normally associated with physical documents which, for example, have signatures, dates; need protection from disclosure, tampering, or destruction; be notarized or witnessed; be recorded or licensed.

- **Authentication** - assurance that the communicating entity is the one claimed
- **Access Control** - prevention of the unauthorized use of a resource
- **Data Confidentiality** –protection of data from unauthorized disclosure
- **Data Integrity** - assurance that data received is as sent by an authorized entity
- **Non-Repudiation** - protection against denial by one of the parties in a communication

Security Services

1. Access control
2. Confidentiality (Secrecy)
3. Integrity
4. Authentication
5. Availability
6. Non-repudiation

Security Goals: CIA Security Triad



- Access control (prevent misuse of resources)
- Confidentiality (privacy)
- Integrity (has not been altered)
- Authentication (who created or sent the data)
- Availability (permanence, non-erasure)
 - Denial of Service Attacks
 - Virus that deletes files
- Non-repudiation (the order is final)

Security Mechanisms

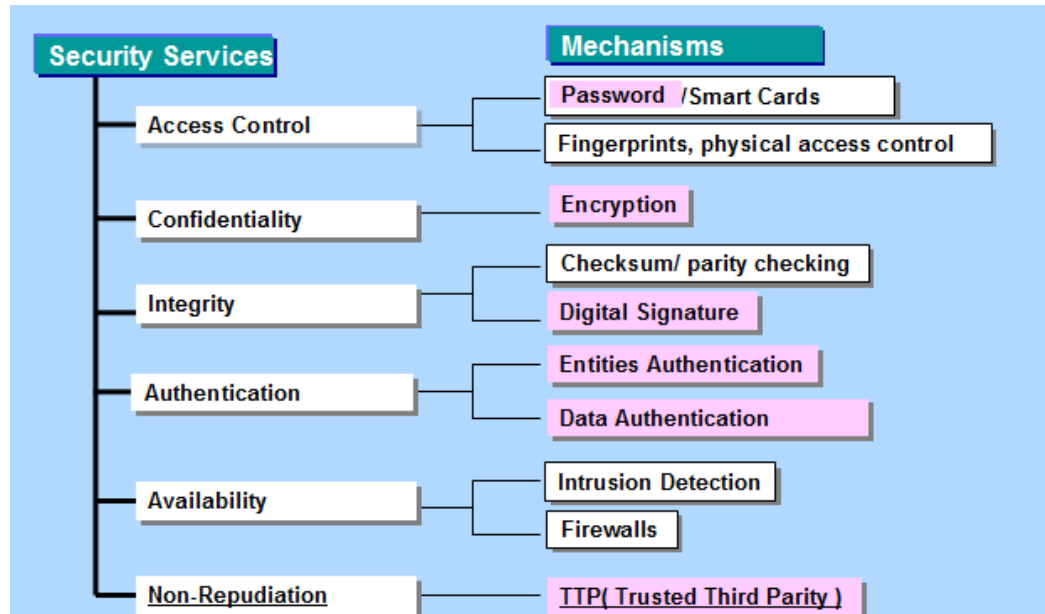
- **Security Mechanism:** A mechanism that is designed to detect, prevent, or recover from a security attack.
- It is a feature designed to detect, prevent, or recover from a security attack
- There is no single mechanism that will support all services required
- however one particular element underlies many of the security mechanisms in use:
 - cryptographic techniques
- hence our focus on this course

- specific security mechanisms:
 - decipherment, digital signatures, access controls, data integrity, authentication exchange, traffic padding, routing control, notarization
 - Encryption
 - Software Controls (access limitations in a data base, in operating system protect each user from other users)
 - Hardware Controls (smartcard)
- pervasive security mechanisms:
 - trusted functionality, security labels, event detection, security audit trails, security recovery
- Policies (frequent changes of passwords)
- Physical Controls

Security Mechanisms

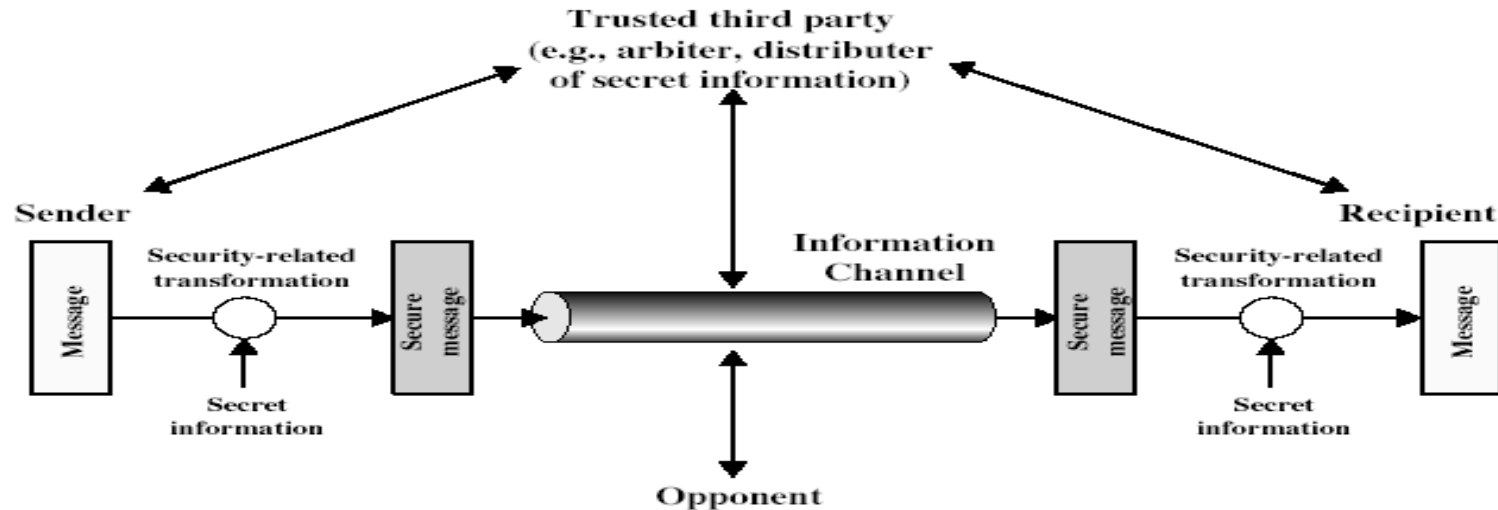
Definition

- ❑ A mechanism that is designed to detect, prevent, or recover from a security attack.



Model for Network Security

- using this model requires us to:
 - design a suitable algorithm for the security transformation
 - generate the secret information (keys) used by the algorithm
 - develop methods to distribute and share the secret information
 - specify a protocol enabling the principals to use the transformation and secret information for a security service



- Using this model requires us to:
 - select appropriate gatekeeper functions to identify users
 - implement security controls to ensure only authorised users access designated information or resources.
 - Trusted computer systems may be useful to help implement this model

Network Access Security Model

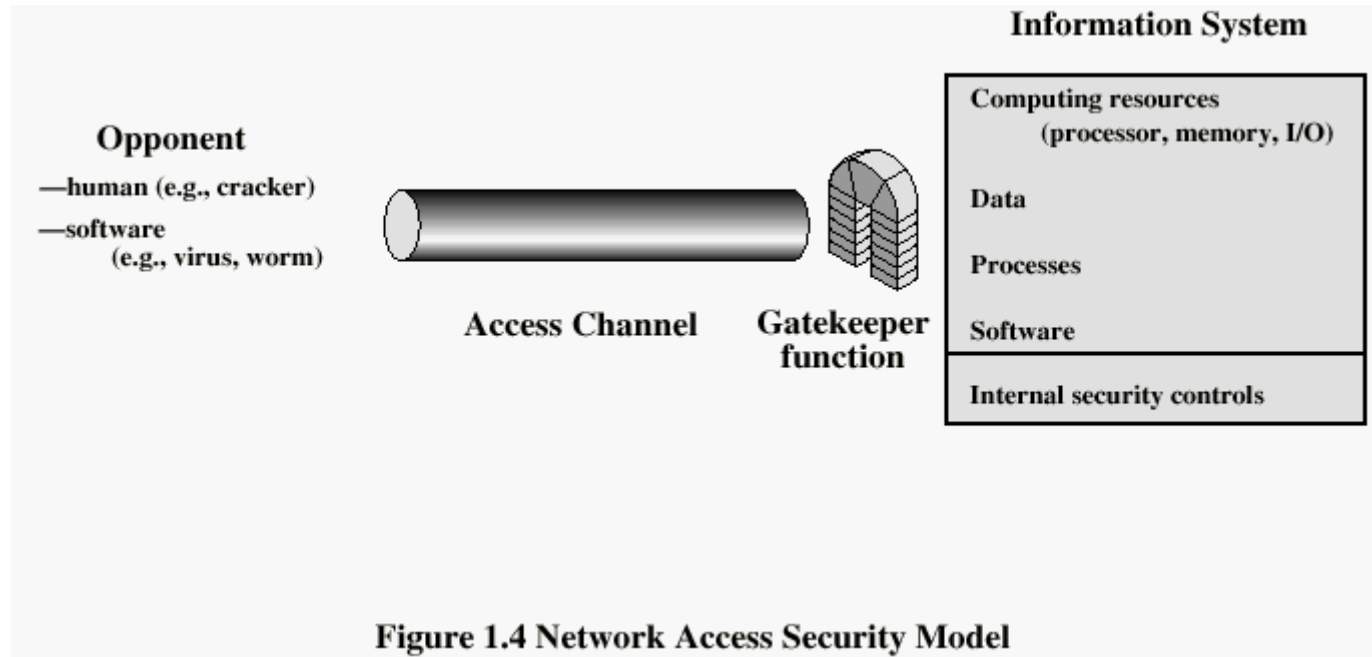


Figure 1.4 Network Access Security Model

Network Access Security Model:

1. Economy of mechanism
2. Fail-safe defaults
3. Complete mediation
4. Open design
5. Separation of privilege
6. Least privilege
7. Least common mechanism
8. Psychological acceptability
9. Isolation
10. Encapsulation
11. Modularity
12. Layering
13. Least astonishment

1.Economy of mechanism

- Keep **designs simple and small**: This makes testing and verification easier. They have fewer weaknesses, need less maintenance, and are easier to update.
- Improves security management – Reducing complexity minimizes configuration issues.
- **Security measures** in both **hardware and software** should be small and straightforward in design.
- Simple designs have fewer weaknesses, require less maintenance, and make updates or replacements easier.
- Challenge of maintaining simplicity: New features often increase complexity, making security harder.
- **Complexity** leads to vulnerabilities: More complex systems have higher security risks. i.e. **complex system** increases the chances of hidden vulnerabilities that attackers can exploit.
- Best practice – Avoid unnecessary complexity in design to ensure better security.
- **more complex** a mechanism is, the **higher** the risk of security flaws.
- They also reduce configuration issues, making security management **more efficient**.

2.Fail-safe default Principle

- **Definition:** Access decisions should be based on permission rather than exclusion.
- The fail-safe default principle ensures that systems are secure by default, minimizing the risk of accidental or malicious access due to design or implementation errors.
- **Default Setting:** The default state is lack of access, and access is granted only under specific conditions.
- **Failure Mode:** This approach ensures a safer failure mode compared to systems where the default is to permit access.

Why Fail-Safe Default is Safer?

■ **Permission-Based Systems:**

- If a design or implementation mistake occurs, the system tends to fail by refusing permission.
- This is a safe failure because it prevents unauthorized access.
 - ✓ Example: A user is denied access to a file due to a misconfiguration, which can be quickly detected and corrected.

■ **Exclusion-Based Systems:**

- If a design or implementation mistake occurs, the system tends to fail by allowing access.
- This is a dangerous failure because unauthorized access may go unnoticed for a long time.
 - ✓ Example: A misconfigured firewall might accidentally allow access to restricted network resources, creating a security breach.

3. Complete Mediation:

- **Every access must be checked.**
 - The system should verify permissions **every time** a user tries to access a resource.
 - It should not rely on **cached** (previously stored) access decisions.
- **Security risks with cached access decisions:**
 - If the system **remembers** access decisions, it must ensure **any changes in permissions** are updated immediately.
 - Otherwise, a user may **retain access** even after their permissions have been revoked.
- **Example – File Access Systems:**
 - When a user **opens a file**, their access is checked.
 - However, if their **permissions change** afterward, the system **does not check again**—this violates complete mediation.
- **Challenges of Full Implementation:**
 - To fully apply this principle, the system would need to **re-check permissions** every time a user reads a file, database entry, or record.
 - This would be **resource-intensive** and is rarely done in practice.

4. Open Design Principle:

- Security mechanisms should be open, not secret.
 - The design of security systems should be transparent and open to evaluation.
 - Open designs increase trust in security mechanisms.
 - Well-tested public algorithms are more secure than proprietary, secret algorithms.
- Encryption algorithms should be public, but keys must remain secret.
 - Secrets should be in the keys, not in the algorithm itself.
 - Example: Advanced Encryption Standard (AES) is a publicly known algorithm, but encryption keys remain private.
- Public scrutiny improves security.
 - When many experts can review an algorithm, they can identify weaknesses and improve security.
- NIST promotes open security standards.
 - Widely accepted encryption standards come from this approach.
 - Example: The National Institute of Standards and Technology (NIST) develops and standardizes encryption and hash algorithms through public evaluation.
 - Example: NIST-approved algorithms like AES, SHA-256, and RSA are globally trusted and widely used in cybersecurity.

5. Separation of Privilege:

- **Multiple privilege attributes are required for access.**
 - Access to a **restricted resource** should require more than one privilege or authentication factor.
 - **Example: Multifactor Authentication (MFA).**
 - ❖ **A user needs multiple authentication methods**, such as:
 - ✓ **Password** (something they know).
 - ✓ **Smart card or fingerprint** (something they have).
- **Programs should be divided based on privilege levels.**
 - Each part of a program should only have the **minimum privileges necessary** for its task.
 - This **reduces security risks** if an attacker compromises one part of the system.i.e. **Limits damage** from security attacks.
 - **Example: High-privilege operations are isolated.**
 - ✓ **Critical functions** (such as system updates or kernel access) should run in a **separate process** with higher privileges.
 - ✓ **Day-to-day tasks** (like web browsing or document editing) should run with **lower privileges** to minimize risk.

6. Least Privilege:

- **Users and processes should have only the minimum privileges required.**
 - No user or program should have more permissions than necessary for their task.
 - **Example: Role-Based Access Control (RBAC).**
 - ✓ Users are assigned **specific roles** with only the **permissions needed** for their tasks.
 - ✓ **Example:** An accountant has access to financial records but not system configurations.
- **Access should be explicitly granted.**
 - Users or processes **cannot access** resources unless **explicit permission** is given.
 - **Example:** A regular employee **cannot edit HR records** unless granted permission.
- **Time-limited privileges prevent misuse.**
 - **Special privileges** should be granted **only when needed** and removed afterward.
 - **Example:** A system administrator should have **elevated access** only when performing maintenance, not during daily tasks.
- **Why It Matters?**
 - **Prevents security breaches** by limiting potential damage.
 - **Reduces accidental errors** caused by excessive privileges.
 - **Enhances system security** by enforcing strict access control.

7. Least Common Mechanism

- **Minimize shared resources between users.**
 - Reduce **common functions** that multiple users depend on to enhance security.
- **Reduces unintended communication paths.**
 - **Limits interactions** between users that could lead to **unauthorized data access**.
- **Minimizes reliance on shared hardware/software.**
 - **Fewer shared components** mean fewer potential security risks.
 - **Example:** Instead of using a **single shared database**, users are given **separate database instances** to prevent data leaks.
- **Easier to verify security risks.**
 - With **fewer shared mechanisms**, it is easier to **audit and detect vulnerabilities**.
 - **Example:** A **dedicated authentication server** for each department is more secure than a single system handling all users.
- **Why It Matters?**
 - **Prevents data leaks** through unintended shared access.
 - **Simplifies security management** by reducing dependencies.
 - **Strengthens system isolation**, reducing security risks.

8. Psychological Acceptability

- **Security should not disrupt user workflow.**
 - Security mechanisms must be **easy to use** and should not create unnecessary obstacles for users.
- **Users may bypass complex security.**
 - If security **slows down work** or is **too difficult**, users may **disable or ignore** it.
 - **Example:** Employees writing down passwords because **complex password policies** are hard to remember.
- ****Security should be transparent or minimally intrusive.**
 - Users should **not feel burdened** by security processes.
 - **Example:** Single Sign-On (SSO) allows users to log in **once** instead of entering passwords multiple times.
- **Security should match the user's mental model.**
 - If security **feels confusing or unintuitive**, users may **make mistakes**.
 - **Example:** If users **expect** file encryption to work **automatically**, but instead need to **manually activate it**, they might forget, leading to security risks.
- **Why It Matters?**
 - **Improves compliance** – Users are more likely to follow security rules.
 - **Reduces errors** – If security makes sense to users, they are less likely to make mistakes.
 - **Enhances productivity** – Well-designed security **protects** without disrupting work.

9.Isolation Principle:

- Public systems should be separated from critical resources.
 - Prevents unauthorized access, data leaks, or tampering.
 - Example: A bank's public website is isolated from its internal transaction system to prevent cyberattacks.
- Sensitive data should be stored on isolated systems.
 - High-security data should be limited to a few, well-protected systems.
 - Example: Military or government systems may use air-gapped networks (physically disconnected from the internet) to prevent cyber threats.
- Logical isolation should include multiple security layers.
 - Firewalls, network segmentation, and access controls should be used between public and secure systems.
 - Example: An organization uses VPNs and multi-layered firewalls to protect internal servers from public-facing networks.

9.Isolation Principle:

- Users' processes and files should be isolated from one another.
 - Prevents unauthorized access between different users.
 - Example: Operating systems like Windows and Linux provide separate memory and file spaces for each user to prevent data leaks.
- Security mechanisms should be isolated from unauthorized access.
 - Critical security functions, like cryptographic operations, should be protected.
 - Example: Encryption software and keys should be stored in secure environments (e.g., Hardware Security Modules (HSMs)) to prevent tampering.
- Why It Matters?
 - Enhances security by minimizing risks of breaches.
 - Reduces attack surfaces by keeping critical functions separate.
 - Prevents data leaks and unauthorized modifications.

10.Encapsulation

- **Encapsulation is a form of isolation in object-oriented security.**
 - Protects **procedures and data objects** by keeping them **within a controlled environment**.
- **Access is restricted to specific procedures.**
 - Only **authorized functions** within the system can access **internal data**.
 - **Example:** In **Java**, a **private variable** can only be accessed through **getter and setter methods**.
- **Limits direct access to internal structures.**
 - Prevents **unauthorized modification** of protected data.
 - **Example:** A **banking system** encapsulates user account details, allowing only approved functions (e.g., deposit, withdraw) to interact with the data.
- **Encapsulation enforces controlled entry points.**
 - Data and functions **cannot be accessed directly** but only through **specific methods**.
 - **Example:** In **API security**, endpoints are defined to control how external systems interact with internal data.
- **Why It Matters?**
 - **Enhances security** by limiting direct access.
 - **Prevents accidental or malicious data changes.**
 - **Improves system integrity** by ensuring only designated functions can modify data.

11.Modularity in Security

- **Procedures should only be accessed through designated entry points.**
 - Ensures **controlled access** to security functions.
 - **Example:** In an **operating system**, only authorized system calls can interact with protected resources.
- **Security functions should be developed as separate modules.**
 - **Each security function should be isolated** to prevent tampering and vulnerabilities.
 - **Example: Cryptographic functions** should be implemented as **separate security modules** rather than in every application.
- **Common security services improve security and efficiency.**
 - **A centralized security module** prevents duplication and enhances security.
 - **Example:** Multiple applications can **use a shared encryption module** instead of implementing their own encryption methods.
- **A modular architecture allows system upgrades without full redesign.**
 - Security mechanisms should support **easy upgrades** and **new technology migration**.
 - **Example:** A **firewall system** should allow **rule updates** without modifying the entire network infrastructure.
- **Why It Matters?**
 - **Reduces complexity** by isolating security mechanisms.
 - **Enhances security** by focusing on securing individual modules.
 - **Improves flexibility** by allowing security upgrades without affecting the whole system.

12.Layering (Defense in Depth)

- **Uses multiple layers of security for protection.**
 - Security should address **people, technology, and operations** to provide comprehensive protection.
- **Prevents a single point of failure.**
 - If **one security measure fails**, others remain in place to protect the system.
 - **Example:** If an attacker bypasses a **firewall**, an **intrusion detection system (IDS)** can still detect and stop the threat.
- **Provides multiple barriers between attackers and sensitive information.**
 - Each layer **adds an extra hurdle** for adversaries.
 - **Example:** A **secure login system** may require:
 - ❖ A **password** (first layer).
 - ❖ **Two-factor authentication (2FA)** (second layer).
 - ❖ **Access control restrictions** (third layer).
- **Commonly referred to as “Defense in Depth.”**
 - Ensures that security is **not reliant on a single mechanism** but **multiple overlapping protections**.
 - **Example:** **Corporate networks** use firewalls, antivirus software, and encryption together for **multi-layered protection**.
- **Why It Matters?**
 - **Increases resilience** against cyber threats.
 - **Reduces attack success rates** by making breaches more difficult.
 - **Ensures continuous protection**, even if one layer fails.

13. Least Astonishment

- **A system should behave as users expect.**
 - The interface and security mechanisms should be **intuitive** and not cause confusion.
- **Authorization processes should be clear and understandable.**
 - Users should **easily grasp how security works** without needing deep technical knowledge.
- **Prevents unexpected or confusing security behaviors.**
 - **Example:** A **password reset system** should clearly indicate password requirements, rather than rejecting a password **after submission** without explanation.
- **Users should feel confident in how security mechanisms work.**
 - Security should **align with user expectations** to avoid mistakes or security bypasses.
 - **Example:** A **banking app** should display a clear **confirmation message** before transferring funds to prevent accidental transactions.
- **Why It Matters?**
 - **Reduces user errors** by making security more predictable.
 - **Improves usability and compliance** since users trust the system.
 - **Enhances security** by preventing confusion that could lead to vulnerabilities.

Any Questions